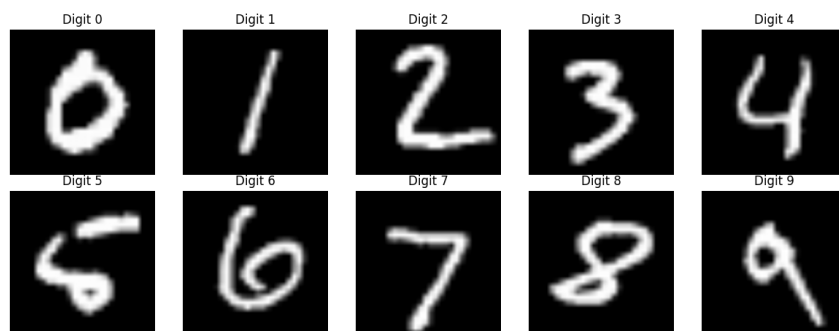


## 1. Zbiory danych

### 1.1. Charakterystyka zbioru MNIST

Pierwszym obiektem, na którym przeprowadzono badania jest popularny w uczeniu maszynowych zbiór danych MNIST (ang. *Modified National Institute of Standards and Technology*). Składający się łącznie około 70 000 obrazów (60 000 przykładów testowych oraz około 10 000 przykładów treningowych), przedstawia ręcznie pisane cyfry od 0 do 9. Każdy z obrazów domyślnie posiada rozmiar 28x28 pikseli, co w efekcie daje 784 piksele (cechy) dla każdego rozważanego obrazu. Cały zbiór danych reprezentowany jest w skali szarości, co oznacza, że każdy piksel przyjmuje wartość z zakresu od 0 do 255, gdzie 0 oznacza kolor czarny, natomiast 255 kolor biały. Przykładowe obrazy z tego zbioru danych przedstawiono na rysunku 1.1.



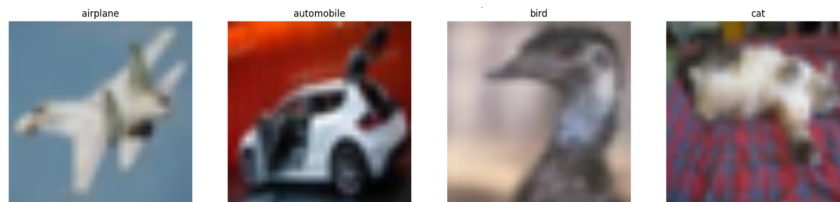
Rysunek 1.1. Przykładowe obrazy z zbioru MNIST

Cechą zbioru MNIST jest jego zbalansowany charakter, co oznacza, że każda z etykiet jest reprezentowana przez podobną ilość przykładów uczących w zbiorze. Zbalansowany zbiór danych jest kluczowy w zadaniach klasyfikacji, ponieważ pozwala na uniknięcie problemu tendencyjności modelu podczas późniejszej klasyfikacji nowych danych, których model wcześniej nie widział. Zdecydowano się na wykorzystanie opisywanego zbioru ze względu na jego popularność i prostotę. Skala szarości, która przekłada się na prostotę zbioru danych pozwala na wstępne zapoznanie się ze specyfiką wyników generowanych przez badane algorytmy wyjaśnialnej sztucznej inteligencji, co stanowi dobrą bazę do dalszych eksperymentów z bardziej złożonymi zbiorami danych.

### 1.2. Charakterystyka zbioru CIFAR-10

Kolejnym zbiorem danych, który wykorzystano w badaniach jest również popularny zbiór CIFAR-10 (ang. *Canadian Institute for Advanced Research*). Zbiór ten charakteryzuje się podobną ilością przykładów, co omawiany wcześniej MNIST. Podobnie jak poprzednik, CIFAR-10 posiada 10 klas, jednak w przeciwieństwie do MNIST, klasy te reprezentują różne

obiekty, takie jak samochody, samoloty, ptaki etc. Od poprzednika różni się również rozmiarem obrazów. W tym przypadku mamy do czynienia z danymi o rozmiarze 32x32 pikseli, co daje 1024 cechy. Z racji, że CIFAR-10 jest zbiorem danych kolorowych, liczbę tę należy pomnożyć przez 3 (kanały RGB), co daje łącznie 3072 cechy dla każdego obrazu. Przykładowe obrazy z tego zbioru danych przedstawiono na rysunku 1.2.



Rysunek 1.2. Przykładowe obrazy z zbioru CIFAR-10

Analogicznie jak w przypadku MNIST, CIFAR-10 jest zbiorem danych zbalansowanym, co oznacza, że każda z klas jest reprezentowana przez podobną ilość przykładów uczących. Wybór omawianego zbioru danych był podyktowany potrzebą zbadania algorytmów wyjaśnialnej sztucznej inteligencji w warunkach danych bardziej złożonych, które odzwierciedlają realistyczne scenariusze (różne gatunki zwierząt, obiekty, tło itp.). Ze względu na swoją różnorodność stanowi doskonały test dla badanych algorytmów, pozwalając na ocenę ich skuteczności w kontekście danych odzwierciedlających świat rzeczywisty.

## 2. Środowisko badawcze

W niniejszym rozdziale przedstawiono wszystkie elementy środowiska badawczego, które zostały wykorzystane w przeprowadzonych eksperymentach. Opisano przygotowanie konfiguracji, danych, modeli klasyfikacyjnych, metod wyjaśniania decyzji modeli, metod generowania perturbacji oraz metryki oceny wyjaśnień. W całym eksperymencie zdecydowano się na podejście obiektowe i modułowe, co oznacza, że każdy z elementów środowiska badawczego został przygotowany w sposób niezależny, a następnie połączony w jedną całość. Takie podejście pozwoliło na elastyczność i możliwość łatwej modyfikacji poszczególnych elementów w przypadku potrzeby modyfikacji parametrów eksperymentu lub wprowadzenia nowych metod wyjaśniania decyzji modeli. W kolejnych sekcjach przedstawiono szczegółowe informacje dotyczące każdego z elementów środowiska badawczego, co pozwoli na pełne zrozumienie procesu przeprowadzonych eksperymentów oraz interpretację uzyskanych wyników.

### 2.1. Przygotowanie konfiguracji, ścieżek oraz funkcji pomocniczych

Pracę nad środowiskiem badawczym rozpoczęto od zaimplementowania zestawu klas konfiguracyjnych oraz zestawu funkcji pomocniczych, których zadaniem było uporządkowanie parametrów eksperymentów i ograniczenie powtarzania tych samych fragmentów kodu w różnych częściach programu.

Do przechowywania parametrów wykorzystano klasy danych oznaczone dekoratorem `@dataclass`. Klasa `IGConfig` zawierała ustawienia metody Integrated Gradients, takie jak liczba kroków całkowania `n_steps` oraz wartość bazowa `baseline_value`. Wartość bazowa była wykorzystywana podczas tworzenia tensora odniesienia dla analizowanego obrazu. Klasa `OcclusionConfig` przechowywała parametry metody Occlusion. Zawierała osobne ustawienia dla zbiorów MNIST oraz CIFAR-10, obejmujące krok przesuwania okna `strides` oraz rozmiar okna okluzji `sliding_window_shapes`. Rozdzielenie tych parametrów pozwalało dostosować działanie metody do charakterystyki analizowanego zbioru danych. W klasie tej zdefiniowano również wartość bazową wykorzystywaną podczas zastępowania fragmentów obrazu.

Klasa `ExperimentConfig` grupowała parametry związane z przebiegiem eksperymentów. Obejmowała wartości odchylenia standardowego szumu Gaussa, wartości parametru `epsilon` dla ataku FGSM, procent pikseli uwzględnianych w metryce Top-K IoU, liczbę kroków w procedurze Deletion AUC oraz wartość bazową stosowaną podczas usuwania pikseli. Dzięki temu wszystkie najważniejsze parametry eksperymentalne były zebrane w jednym miejscu i mogły być łatwo modyfikowane bez ingerencji w kod poszczególnych klas. Klasa `TrainingConfig` zawierała podstawowe ustawienia procesu uczenia modelu, takie jak rozmiar batcha, liczba epok, współczynnik uczenia oraz udział zbioru walidacyjnego. Parametry te były

wykorzystywane podczas przygotowywania danych oraz trenowania modeli klasyfikacyjnych. W projekcie zdefiniowano również klasę `Paths`, której zadaniem było przechowywanie ścieżek do najważniejszych katalogów projektu. Na podstawie lokalizacji pliku wyznaczano katalog główny projektu, a następnie katalog danych oraz katalog przeznaczony do zapisu modeli.

Dodatkowe funkcje pomocnicze umieszczono w pliku `utils.py`. Funkcja `get_device()` automatycznie wybierała urządzenie obliczeniowe, zwracając GPU w przypadku dostępności technologii CUDA albo CPU w przeciwnym przypadku. Funkcja `tensor_to_img()` służyła do przekształcania tensora obrazu do postaci możliwej do wyświetlenia. W ramach tej operacji tensor był przenoszony na CPU, konwertowany do tablicy NumPy, a następnie przekształcany z zakresu wartości  $[-1,1]$  do zakresu  $[0,1]$ . Funkcja `plot_image()` ujednolicała sposób wyświetlania obrazów i map atrybucji. Przyjmowała obiekt osi wykresu, obraz, tytuł oraz opcjonalne parametry mapy kolorów i zakresu wartości. Dzięki temu możliwe było stosowanie spójnego sposobu wizualizacji w różnych częściach eksperymentu. W pliku pomocniczym zdefiniowano także funkcję `initialize_resnet18()`, która tworzyła model ResNet18. W zależności od przekazanego parametru model mógł zostać zainicjalizowany z wagami wstępnie wytrenowanymi lub bez nich. Ostatnia warstwa była zastępowana nową warstwą, której liczba wyjść odpowiadała liczbie klas analizowanego zbioru danych. Funkcja `load_model_weights()` umożliwiała wczytanie zapisanych wag modelu, przeniesienie modelu na odpowiednie urządzenie oraz przełączenie go w tryb ewaluacji.

## 2.2. Przygotowanie danych oraz procesu trenowania

Następnie przystąpiono do przygotowania klasy `DataManager`, która służyła do ujednolicenia procesu przygotowania danych wykorzystywanych w badaniach. Jej głównym zadaniem było zapewnienie wspólnego interfejsu do obsługi zbioru MNIST oraz CIFAR-10, obejmującego pobieranie danych, ich wstępne przetwarzanie, wyodrębnienie ze zbioru treningowego zbioru walidacyjnego, pobranie zbioru testowego oraz obiektów `Data Loader`, które umożliwiały iterację po danych w trakcie trenowania modeli. Dzięki temu możliwe było łatwe zarządzanie danymi oraz zapewnienie spójności w procesie przygotowania danych dla eksperymentów. W konstruktorze klasy określano ścieżkę do katalogu danych na podstawie klasy `Paths` oraz tworzonego obiektu `TrainingConfig`, z którego pobierano parametry związane z treningiem, takie jak rozmiar batcha oraz udział zbioru walidacyjnego. Klasa przechowywała również mapowanie indeksów klas na ich nazwy. Dla zbioru MNIST indeksy odpowiadały cyfrom od 0 do 9, natomiast dla zbioru CIFAR-10 przypisano im nazwy klas, takie jak `airplane`, `automobile`, `bird` czy `cat`. Metoda `get_class_names()` umożliwiała pobranie tego mapowania dla wskazanego zbioru danych.

Istotnym elementem klasy `DataManager` jest metoda `get_transform()`, która definiuje sposób przetwarzania obrazów w zależności od zbioru danych. Obrazy są przekształcane do rozmiaru 224x224 pikseli, następnie zamieniane na obiekty typu `tensor`. Ostatnim

etapem przygotowywania danych dla modelu jest normalizacja. W badaniach została ona przeprowadzona zgodnie ze wzorem 2.1.

$$x_{norm} = \frac{x - \mu}{\sigma} \quad (2.1)$$

gdzie:

$x$  – wartość wejściowa piksela,

$\mu$  – średnia wartość pikseli dla danego zbioru,

$\sigma$  – odchylenie standardowe wartości pikseli.

Wartości  $\mu$  i  $\sigma$  zostały przyjęte na poziomie 0.5. Podstawiając te wartości do wzoru 2.1 otrzymujemy:

$$x_{norm} = \frac{x - 0.5}{0.5} = 2x - 1 \quad (2.2)$$

Po transformacji obrazów do obiektu typu tensor wartości pikseli zawierają się w przedziale  $[0, 1]$ . Podstawiając te wartości do wzoru 2.2 otrzymujemy, że po normalizacji wartości pikseli zawierają się w przedziale  $[-1, 1]$ . Taka normalizacja pozwala na wycentrowanie danych wejściowych wokół zera, co może prowadzić do stabilniejszego przebiegu procesu uczenia oraz efektywniejszej optymalizacji parametrów modelu. Należy również uwzględnić dodatkowy krok transformacyjny w przypadku zbioru MNIST. Ponieważ obrazy w tym zbiorze są jednokanałowe (skala szarości), konieczne było dodanie dodatkowego kroku transformacyjnego, który rozszerza pojedynczy kanał do trzech kanałów, ponieważ taki właśnie format wymagany jest przez architekturę sieci użytej w badaniach.

Metoda `get_dataset()` pozwala na utworzenie odpowiedniego obiektu zbioru danych z biblioteki PyTorch, nakładając jednocześnie zdefiniowane wcześniej kroki transformacyjne. W celu wydzielenia zbioru walidacyjnego służącego do oceny modeli podczas procesu uczenia zaimplementowana została metoda `get_train_val_datasets()`. Omawiana metoda pobiera pełny zbiór treningowy i wydobywa z niego zbiór walidacyjny o wielkości determinowanej przez parametr `val_ratio`, który przyjęto na poziomie 0.1. Metoda `get_test_dataset()` stanowi wygodny sposób na pobranie zbioru testowego dla danego zbioru danych nakładając na niego odpowiednie transformacje. Ostatnią główną funkcjonalnością klasy `DataManager` jest metoda `get_data_loaders()`, która tworzy obiekty `DataLoader` dla zbioru treningowego, walidacyjnego oraz testowego. Obiekty te umożliwiają iterację po danych w trakcie trwania procesu uczenia modeli.

Po przygotowaniu klasy odpowiedzialnej za zarządzanie danymi przystąpiono do implementacji modułu `ModelTrainer`, którego zadaniem było uporządkowanie procesu treningu, walidacji oraz ewaluacji modeli klasyfikacyjnych wykorzystywanych w badaniach. Klasa ta pozwalała oddzielić logikę uczenia modelu od pozostałych elementów projektu.

Obiekt klasy `ModelTrainer` przyjmował model sieci neuronowej, urządzenie wykonujące obliczenia oraz nazwy klas występujących w analizowanym zbiorze danych. W konstruktorze określano również ścieżkę zapisu modelu na podstawie klasy `Paths` oraz tworzonego obiektu `TrainingConfig`, z którego pobierano parametry konieczne do przeprowadzenia treningu. Funkcja straty została zdefiniowana jako `CrossEntropyLoss`, natomiast jako optymalizator wykorzystano algorytm Adam. Model był następnie przenoszony na wskazane urządzenie obliczeniowe. Klasa przechowywała również historię procesu uczenia w postaci słownika zawierającego wartości funkcji straty oraz dokładności dla zbioru treningowego i walidacyjnego. Dzięki temu możliwa była późniejsza analiza przebiegu uczenia modelu. Metoda `train_one_epoch()` realizowała pojedynczą epokę treningową. Dla kolejnych batchy wykonywana była propagacja w przód, obliczanie funkcji straty, propagacja wsteczna oraz aktualizacja wag modelu. Dla każdej epoki obliczana była średnia wartość funkcji straty oraz dokładność klasyfikacji na zbiorze treningowym. W metodzie `validate()` dokonywana była ocena modelu na zbiorze walidacyjnym w danej epoce. Wynikiem działania metody były średnia strata walidacyjna oraz dokładność klasyfikacji na zbiorze walidacyjnym. Metoda `fit()` pełniła rolę orkiestratora dwóch poprzednich metod, wywołując je w pętli po liczbie zadanych epok treningowych. Wyniki metod `train_one_epoch()` i `validate()` zapisywane były w historii treningu. Informacje w niej zawarte, czyli dokładność oraz wartości funkcji straty, były wykorzystywane do oceny jakości procesu uczenia. Po zakończeniu treningu model mógł zostać oceniony na zbiorze testowym za pomocą metody `evaluate()`, która generowała predykcje dla wszystkich przykładów testowych oraz raport klasyfikacyjny zawierający metryki dla poszczególnych klas. Ostatnią funkcjonalnością klasy `ModelTrainer` była metoda `save_model()`, która umożliwiała zapis wag modelu do pliku, dając możliwość załadowania i wykorzystania modelu w późniejszych badaniach.

### 2.3. Metody generowania perturbacji

W celu generowania zakłóconych wersji obrazów wejściowych wykorzystywanych w eksperymentach nad stabilnością metod XAI zaimplementowano klasę `Perturbations`. Jej głównym zadaniem jest przygotowanie danych poddanych perturbacjom, a następnie wykorzystanie ich do porównania map atrybucji uzyskanych dla obrazów oryginalnych i zaburzonych.

Obiekt klasy `Perturbations` przyjmuje model predykcyjny, urządzenie obliczeniowe oraz wartości ograniczające zakres pikseli po perturbacji. Zgodnie z opisem procesu przygotowania danych w sekcji 2.2, dane wejściowe zostały znormalizowane do zakresu  $[-1, 1]$ , zatem również takie wartości przyjęto jako domyślne w parametrach, zapewniając, że po dodaniu zakłócenia obraz nadal pozostaje w dopuszczalnym zakresie wartości wejściowych modelu. Jest to istotne, ponieważ zbyt duże perturbacje mogłyby przesunąć wartości pikseli poza zakres danych, na których model był trenowany.

Pierwszą zaimplementowaną metodą perturbacji jest dodanie szumu Gaussa, który zapewnia metoda `gaussian()`. Tworzy ona losowy szum o takim samym rozmiarze jak tensor wejściowy, a następnie skaluje go przez parametr  $\sigma$ , który określa intensywność generowanych zakłóceń. Przygotowany szum jest dodawany do obrazu wejściowego zgodnie ze wzorem 2.3.

$$x_{noise} = x + \mathcal{N}(0, \sigma^2) \quad (2.3)$$

gdzie:

$x$  – wartość wejściowa piksela,

$\mathcal{N}(0, \sigma^2)$  – szum pochodzący z rozkładu Gaussa ze średnią równą 0 i i odchyleniu standardowym  $\sigma$ ,

$\sigma$  – odchylenie standardowe szumu (poziom zakłóceń).

Większa wartość parametru  $\sigma$  oznacza silniejsze zakłócenie dodawane do obrazu. Po dodaniu szumu wartości pikseli są przycinane zgodnie z parametrami `clamp_min` oraz `clamp_max`. Wartości mniejsze od -1 zastępowano wartością -1, natomiast wartości większe od 1 zastępowano wartością 1. Operacja ta zapobiegała pojawianiu się wartości spoza zakresu wykorzystywanego podczas trenowania modelu.

/

## **2.4. Metody wyjaśniania decyzji modeli**

## **2.5. Metryki oceny wyjaśnień**

## **2.6. Konfiguracja eksperymentów**

### **3. Analiza stabilności wyjaśnień dla zbioru MNIST**

#### **3.1. Integrating Gradients w obliczu szumu Gaussa**

#### **3.2. Integrating Gradients w obliczu ataków FGSM**

#### **3.3. Occlusion w obliczu szumu Gaussa**

#### **3.4. Occlusion w obliczu ataków FGSM**



## **4. Analiza stabilności wyjaśnień dla zbioru CIFAR-10**

### **4.1. Integrating Gradients w obliczu szumu Gaussa**

### **4.2. Integrating Gradients w obliczu ataków FGSM**

### **4.3. Occlusion w obliczu szumu Gaussa**

### **4.4. Occlusion w obliczu ataków FGSM**

## **5. Analiza porównawcza uzyskanych wyników**